

```
ui->teDetails->clear();
ui->teDetails->append("Connection is : "+
    model->database().connectionName());
ui->teDetails->append("Database is : "+
    model->database().databaseName());
ui->teDetails->append("Table Name : "+model->tableName());
ui->teDetails->append("Where Clause : "+model->filter());
```

QSqlQueryModel

This is a base class for QSqlTableModel and provides read-only access to the table data:

```
QSqlQueryModel* model = new QSqlQueryModel;
model->setQuery("select * from student");
model->setHeaderData(1, Qt::Horizontal, QVariant("Name"));
```

We use the *setQuery* function instead of *setTable* here. Unlike the table model, we don't need to call the select function explicitly. The data is automatically populated using the *setQuery* call. The *setQuery* function takes the connection object as an optional second argument. We also use the *setHeaderData* function to rename column headers. In this example, *sname* is renamed Name.

We can bind the above example to the table view as well, in a similar fashion.

Other applicable operations for QSqlQueryModel are:

- *clear()* to detach from the table and release all acquired information.
- *removeColumns(fromCol, numCols)* filters a specified number of columns from a given column.
- *record(pos)* returns the QSqlRecord for record at a given position.
- *setHeaderData(index, orientation, value)* renames the row, column headers as per given orientation which is horizontal per columns and vertical per rows.
- *headerData(index, orientation)* fetches the row or column header names.
- *lastError()* returns an error object if there is any failure in the recent operation.

We can use these models without mapping to View.

For example, here's the code to get all data associated with these models:

```
int rollo=QString name;double marks;
for(int i=0;i<model->rowCount();i++)
{
    rollo=model->record(i).value(0).toInt();
    name=model->record(i).value(1).toString();
    marks=model->record(i).value(2).toDouble();
    qDebug() << rollo << " " << name << " " << marks;
}
```

And here's the code to replace or insert a particular record using QSqlTableModel:

```
QSqlRecord rec;
```

```
QSqlField fd1("rollo",QVariant::Int); fd1.setValue("121");
rec.insert(0,fd1);
QSqlField fd2("sname",QVariant::String); fd2.setValue("jkt");
rec.insert(1,fd2);
QSqlField fd3("marks",QVariant::Double); fd3.setValue("89.5");
rec.insert(2,fd3);
model->insertRecord(-1,rec);
model->setRecord(5,rec); // to replace 5th record with above one
```

QSqlError

To ensure any database operation has not failed, the *lastError* function should be called, which returns an object of the QSqlError class. Then, checking the *isValid()* function on that object will indicate whether an error has occurred or not. Some important members of the QSqlError class are:

- *type()* returns the exact type of error—connection error, statement error, transaction error, unknown error, or no error.
- *number()* returns database-specific error numbers.
- *databaseText()* returns the error message as reported by the database.
- *driverText()* returns the error message as reported by the driver.
- *text()* returns the concatenated string of the database and driver error messages.

The following code depicts the usage of the QSqlError function:

```
//some operation with a model or query object
QSqlError err=model.lastError();
if(err.isValid())
{
    qDebug() << err.databaseText();
    qDebug() << err.driverText();
    qDebug() << "Error Type" << err.type();
    qDebug() << "Error no: " << err.number();
    // Do error handling
}
```

In this article, we haven't listed or discussed the details of some members. Referring to the QtSql section of the Qt documentation can help you further. 

References

- Official Qt website: qt.nokia.com
- Documentation for Qt and related products: doc.trolltech.com

By Rajesh Sola

The author is a faculty member of the Computer Science Department at NBKRIST, Vidyanagar. He is a contributor to the OpenOffice.org project and is keen on promoting FOSS awareness and adoption in rural areas. He believes in encouraging and supporting students to take the open source road. You can reach him at rajesh@lisor dot org.